Índice

Resumen **2**

Introducción general **3**

PARTE I — La tecnología: librosa **4**

4.2. ¿Qué es? 4

4.3. ¿Cómo funciona? 5

4.4. ¿Para qué se usa? 13

4.5. Casos de uso reales 13

4.6. Comparación con alternativas 14

4.7. Instalación, ecosistema y buenas prácticas 15

PARTE II — El proyecto: Voicer Choicer en consola **16**

4.8. Descripción introductoria 16

4.9. Estructura del proyecto 17

4.10. Cómo ejecutar el proyecto 18

4.11. Instrucciones de uso 19

4.12. Descripción estructural en profundidad 21

4.13. Decisiones de diseño y trade-offs 29

4.14. Dificultades encontradas y aprendizajes 31

PARTE III — Conclusiones y cierre **33**

4.15. Conclusiones 33

4.16. División del trabajo 33

4.17. Declaración de uso de herramientas de IA 34

4.18. Referencias 34

Anexos **36**

Anexo A. Salida de las pruebas automáticas 36

Anexo B. Constantes de configuración 36

Anexo C. Glosario 37

La carátula (título, materia, integrantes con nombre, legajo y mail, fecha y docentes) se agrega por separado antes de la entrega.

Resumen

Este trabajo investiga **librosa**, una biblioteca de Python para el análisis de señales de audio y música, en su versión 1.0.0 (agosto de 2026). En la Parte I se explica qué es, cómo está organizada y, sobre todo, cómo funciona por dentro. Para eso se leyó el código fuente de las funciones centrales: la carga de audio, la transformada de Fourier de tiempo corto (STFT), los coeficientes cepstrales en frecuencia mel (MFCC), la estimación de tono pYIN, la alineación temporal DTW y el decodificador de Viterbi. Se describen las estructuras de datos que usa (principalmente arreglos contiguos de NumPy y vistas sin copia), la complejidad de cada algoritmo y los mecanismos de Python en los que se apoya: carga diferida de módulos, decoradores de caché y compilación a código máquina con numba.

En la Parte II se documenta el proyecto desarrollado: una versión para consola del juego *Voicer Choicer*. El programa reproduce un clip, de uno a tres jugadores lo imitan por el micrófono y cada imitación recibe un puntaje de 0 a 100. El puntaje combina un 60 % de similitud de timbre, medida con MFCC alineados por DTW, y un 40 % de similitud de melodía, medida con pYIN sobre el mismo camino de alineación. El ranking se guarda en un archivo JSON dentro del proyecto, sin base de datos.

Como conclusión principal, librosa resultó adecuada para el problema. Ofrece en pocas líneas algoritmos que de otro modo habría que implementar desde cero, y su código es legible y está bien documentado. Sus costos son una dependencia pesada (NumPy, SciPy, numba y otras), una primera ejecución lenta por la compilación de numba y la necesidad de conocer cómo funcionan algunas funciones para usarlas bien. El ejemplo más claro es el recorte de silencios, que mide en forma relativa.

Introducción general

La consigna del trabajo pide investigar a fondo una tecnología de Python no vista en clase, aplicarla en un proyecto propio y explicar ambas cosas como si se tratara de una clase para el resto del curso. Elegimos el procesamiento de audio porque es un área en la que las estructuras de datos y los algoritmos de la materia aparecen de forma muy concreta. Un sonido digital es un arreglo de números. Analizarlo implica dividirlo en ventanas, transformarlo y compararlo, y para eso se usan programación dinámica, modelos ocultos de Markov y la transformada rápida de Fourier.

La motivación del proyecto es lúdica: replicar un juego de imitación de voz. Detrás de esa idea simple hay un problema técnico interesante. ¿Cómo decide una computadora si dos personas "dijeron lo mismo de la misma manera", si hablan a distinta velocidad, con distinta altura de voz y por distintos micrófonos? librosa resuelve las partes difíciles de ese problema y por eso fue la elegida.

Objetivos. Comprender la arquitectura y el funcionamiento interno de librosa. Identificar las estructuras de datos y algoritmos que emplea y justificar su elección. Compararla con alternativas. Desarrollar un juego funcional, modular y probado que la use de forma sustancial. Documentar las decisiones de diseño con argumentos.

Organización del informe. La Parte I (secciones 4.2 a 4.7) trata sobre la tecnología: qué es, cómo funciona, para qué se usa, casos reales, alternativas e instalación. La Parte II (4.8 a 4.14) documenta el proyecto: alcance, estructura, ejecución, uso, diseño interno, decisiones y dificultades. La Parte III (4.15 a 4.18) reúne las conclusiones, la división del trabajo, la declaración de uso de herramientas de IA y las referencias. Los anexos contienen la salida de las pruebas y el detalle de la calibración. La numeración de las secciones sigue la de la consigna.

PARTE I — La tecnología: librosa

4.2. ¿Qué es?

librosa es una biblioteca de código abierto escrita en Python que provee los bloques básicos para analizar señales de audio y música. Permite leer archivos de sonido, calcular representaciones tiempo-frecuencia (espectrogramas), extraer características como timbre, tono, ritmo o armonía, detectar eventos (inicios de nota, pulsos) y comparar o segmentar secuencias. Según su propia documentación, su objetivo es dar las herramientas fundamentales para construir sistemas de *Music Information Retrieval* (MIR), el área que estudia cómo extraer información de la música (librosa development team, 2026a).

Contexto y origen

librosa nació en el laboratorio LabROSA de la Universidad de Columbia, del que toma el nombre, y se presentó formalmente en la conferencia SciPy de 2015 (McFee et al., 2015). Su autor principal y actual responsable de mantenimiento es Brian McFee, hoy en la Universidad de Nueva York. El problema que vino a resolver era que buena parte de la investigación en MIR se hacía en MATLAB, con código disperso, difícil de reproducir y sin una interfaz común. librosa propuso reunir esos algoritmos en Python, sobre NumPy y SciPy, con nombres y convenciones uniformes. Los autores expresaron como principios de diseño la consistencia de la interfaz, la compatibilidad hacia atrás, la modularidad y la legibilidad del código, incluso por encima del rendimiento extremo (McFee et al., 2015).

Estado actual

Aspecto	Situación (octubre de 2026)
Versión estudiada	1.0.0, publicada el 11 de agosto de 2026 en PyPI, tras más de diez años en la serie 0.x (librosa development team, 2026b)
Madurez	Clasificada como *Production/Stable*. La 1.0 tiene como objetivo declarado estabilizar la interfaz de la serie 0.x y eliminar las funciones que estaban marcadas como obsoletas
Requisitos	Python 3.12 o superior (la 0.11 aceptaba versiones anteriores)
Licencia	ISC, una licencia permisiva equivalente en la práctica a MIT
Comunidad y mantenimiento	Repositorio público en GitHub con integración continua, documentación en librosa.org, foro de discusión y publicación con firma verificable (*Trusted Publishing*) desde GitHub Actions
Citas	Cada versión tiene DOI en Zenodo; desde la 1.0, `librosa.cite(bib=True)` devuelve la entrada BibTeX de la versión instalada

Tabla 1. Estado de librosa al momento del trabajo.

Posición en el ecosistema

librosa no implementa todo desde cero. Es una capa de alto nivel que combina otras bibliotecas del ecosistema científico de Python. Las dependencias que instala la versión 1.0.0 (verificadas con `pip show librosa`) son:

Dependencia	Para qué la usa librosa
NumPy	Estructura de datos central: toda señal, espectrograma o matriz es un `ndarray`
SciPy	FFT (`scipy.fft.rfft`, `dct`), distancias (`cdist`) y distribuciones (`scipy.stats.beta` en pYIN)
numba	Compila a código máquina los bucles que NumPy no puede vectorizar (DTW, Viterbi)
soundfile	Lee y escribe archivos de audio a través de la biblioteca en C libsndfile
soxr	Remuestreo de alta calidad (cambiar la frecuencia de muestreo)
joblib	Caché en disco opcional de resultados de funciones
scikit-learn	Algoritmos de agrupamiento y descomposición (módulos `segment` y `decompose`)
lazy_loader, decorator, msgpack, pooch	Carga diferida de submódulos, decoradores que preservan firmas, datos internos y descarga de audios de ejemplo

Tabla 2. Dependencias directas de librosa 1.0.0.

Un cambio relevante de la 1.0 es que **ya no depende de audioread**. Las versiones anteriores lo usaban como alternativa para formatos que libsndfile no leía, como MP3 en versiones viejas. En la 1.0, `librosa.load` usa únicamente soundfile (se verificó en `core/audio.py`).

4.3. ¿Cómo funciona?

Arquitectura general

librosa está organizada como un conjunto de submódulos de funciones, no de clases. Casi toda la interfaz son funciones puras que reciben arreglos de NumPy y devuelven arreglos de NumPy, al estilo de SciPy. La Figura 1 muestra los submódulos principales y sus dependencias, reconstruidos a partir de los `import` del código fuente.

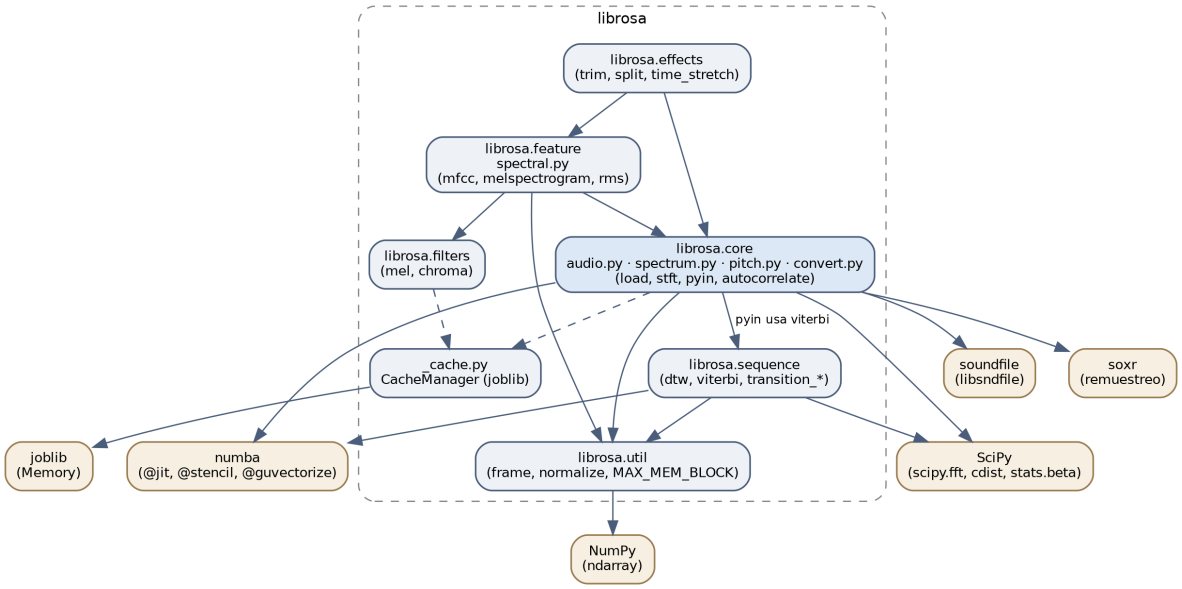


Figura 1. Arquitectura de librosa 1.0.0: submódulos internos (azul) y bibliotecas externas (ocre). Elaboración propia.

- **core**: carga y remuestreo (`audio.py`), espectrogramas (`spectrum.py`), estimación de tono (`pitch.py`) y conversiones de unidades (`convert.py`). Sus funciones se exponen en el nivel superior: `librosa.load`, `librosa.stft`, `librosa.pyin`.
- **feature**: características derivadas del espectrograma, como MFCC, espectrograma mel, croma y energía RMS.
- **filters**: construye bancos de filtros (mel, croma) que se aplican como multiplicaciones de matrices.
- **effects**: operaciones sobre la señal, como recortar silencios, dividir por pausas o cambiar velocidad y altura.
- **sequence**: algoritmos sobre secuencias, entre ellos DTW, Viterbi y matrices de transición.
- **util**: utilidades de bajo nivel, como el entramado en ventanas, la normalización y las constantes de memoria.

Modelo de ejecución

librosa no tiene un "bucle principal" ni un servidor. Cada llamada es **sincrónica**: corre en el hilo que la invoca, procesa el arreglo completo en memoria y devuelve el resultado. Para describir el ciclo de vida de una operación típica seguimos la que más usa nuestro proyecto, que va desde un archivo hasta sus MFCC (Figura 2).



Figura 2. Ciclo de vida de una extracción de MFCC, con las formas de los arreglos intermedios. Elaboración propia a partir del código fuente.

1. **Carga** (`core/audio.py`, función `load`). Se abre el archivo con `soundfile.SoundFile` y se leen las muestras como `float32`. Si es estéreo, `to_mono` promedia los canales. Luego `resample` lleva la señal a 22 050 Hz con soxr en modo `soxr_hq`. El resultado es un vector `y`

de forma (N,).

2. **Entramado** (`util.frame`). La señal se divide en ventanas de 2048 muestras que avanzan de a 512. No se copian datos: `frame` usa `numpy.lib.stride_tricks.as_strided` para crear una **vista** en la que cada columna empieza 512 muestras después que la anterior. En una prueba con 3 s de audio, la vista tenía forma (2048, 126) y pasos de memoria (4, 2048) bytes, y apuntaba al mismo búfer que la señal.
3. **STFT** (`core/spectrum.py`, `stft`). A cada ventana se le aplica una ventana de Hann y una FFT real (`scipy.fft.rfft`). Las columnas se procesan en bloques para no superar `util.MAX_MEM_BLOCK` = 2^{18} bytes (256 KiB) de memoria temporal. El resultado es una matriz compleja de 1025 × T, donde 1025 = 2048/2 + 1 son las frecuencias y T los cuadros.
4. **Espectrograma mel** (`feature/spectral.py`, `melspectrogram`). Se eleva al cuadrado el módulo para obtener la potencia y se multiplica por el banco de filtros de `filters.mel` (128 × 1025) con `np.einsum`. Así 1025 frecuencias lineales se resumen en 128 bandas espaciadas como las percibe el oído.
5. **Escala logarítmica** (`power_to_db`): $10 \cdot \log_{10}$, porque percibimos el volumen en forma aproximadamente logarítmica.
6. **DCT** (`mfcc`). Una transformada discreta del coseno de tipo II (`scipy.fft.dct`) sobre el eje de las bandas descorrelaciona la información y la concentra en los primeros coeficientes. Se conservan los primeros `n_mfcc` (por defecto 20). El resultado es una matriz de n_mfcc × T: cada columna describe la "forma" del espectro en ese instante, que es lo que percibimos como timbre (Davis & Mermelstein, 1980).

Estructuras de datos internas

La estructura central de librosa es el **arreglo multidimensional de NumPy** (`ndarray`). Se trata de un bloque de memoria **contiguo** con datos de un único tipo, acompañado de metadatos: forma, tipo de dato y pasos (*strides*). Que sea la elección obvia se explica comparándolo con una lista de Python:

Criterio	list de Python	ndarray de NumPy
Almacenamiento	Arreglo de punteros a objetos `float` dispersos en memoria	Valores crudos uno al lado del otro
Memoria para 3 s de audio (66 150 muestras)	≈ 2,1 MB (8 B de puntero + 24 B por objeto float)	≈ 265 KB (4 B por muestra en float32)
Acceso por índice	O(1)	O(1), con cálculo directo de dirección
Operación sobre todos los elementos	Bucle interpretado, un objeto por vez	Bucle en C sobre memoria contigua (vectorización), con buen uso de caché y SIMD
Vistas sin copia	No (un *slice* copia)	Sí: *slicing*, `as_strided`, transposición

Tabla 3. Comparación entre list y ndarray para representar señales.

Por convención, librosa ubica **el tiempo en el último eje**. Una señal es (N,), un espectrograma es (frecuencias, T) y los MFCC son (coeficientes, T). Desde la serie 0.9 se aceptan dimensiones extra al principio para audio multicanal: (canales, N). Otras estructuras relevantes:

- **Matriz de costo acumulado** en DTW: un `ndarray` de (N+1) × (M+1) inicializado en infinito, más una matriz paralela de enteros con el paso elegido en cada celda, que sirve para reconstruir el camino.
- **Lista de tuplas** para el camino de DTW. Mientras retrocede, `__dtw_backtracking` agrega pares (i, j) a una lista de Python con `append`, en O(1) amortizado, porque no conoce de antemano el largo del camino. Al final la convierte en arreglo de (L, 2).
- **Matriz de transición** del modelo oculto de Markov de pYIN, construida con el producto de Kronecker (`np.kron`) de una matriz 2 × 2 (con tono / sin tono) y una matriz de bandas que solo permite saltos de tono pequeños entre cuadros.
- **Listas de predecesores** en Viterbi. Para cada estado se precalcula con `np.flatnonzero` qué estados pueden llegar a él con probabilidad no despreciable. Es una representación dispersa, del estilo de las listas de adyacencia de un grafo, que evita recorrer transiciones imposibles.

Algoritmos relevantes

Transformada rápida de Fourier (FFT). Calcular la transformada discreta de una ventana de n muestras por definición cuesta $O(n^2)$. El algoritmo de Cooley y Tukey (1965) aplica *divide y vencerás*: separa las muestras pares de las impares, resuelve recursivamente las dos mitades y combina los resultados, con lo que el costo baja a **O(n log n)**. Para n = 2048 son unas 22 500 operaciones contra 4,2 millones. La STFT completa cuesta O(T · n log n). librosa delega la FFT en `scipy.fft`; la autocorrelación de pYIN también se calcula con FFT (`core/audio.py`, `autocorrelate`), que elige un tamaño rápido con `next_fast_len`.

Dynamic Time Warping (DTW) (Sakoe & Chiba, 1978; Müller, 2015, cap. 3). Sirve para comparar dos secuencias que representan "lo mismo" a distinta velocidad. Es **programación dinámica**, como el problema de la distancia de edición entre cadenas:

1. Se calcula la matriz de costos locales C[i, j] = distancia euclídea entre la columna i de X y la columna j de Y, con `scipy.spatial.distance.cdist`, en O(N·M·d).
2. Se llena la matriz de costo acumulado con la recurrencia D[i, j] = C[i, j] + mín(D[i−1, j−1], D[i, j−1], D[i−1, j]). Los pasos permitidos por defecto son (1,1), (0,1) y (1,0) (`default_steps` en `sequence.py`). Esto lo hace `__dtw_calc_accu_cost`, un doble bucle compilado con numba, en **O(N·M)** de tiempo y memoria.
3. Se reconstruye el camino óptimo desde la esquina (N, M) hasta (0, 0) siguiendo la matriz de pasos guardada (`__dtw_backtracking`), en O(N + M). Por eso el camino se devuelve **del final al principio**.

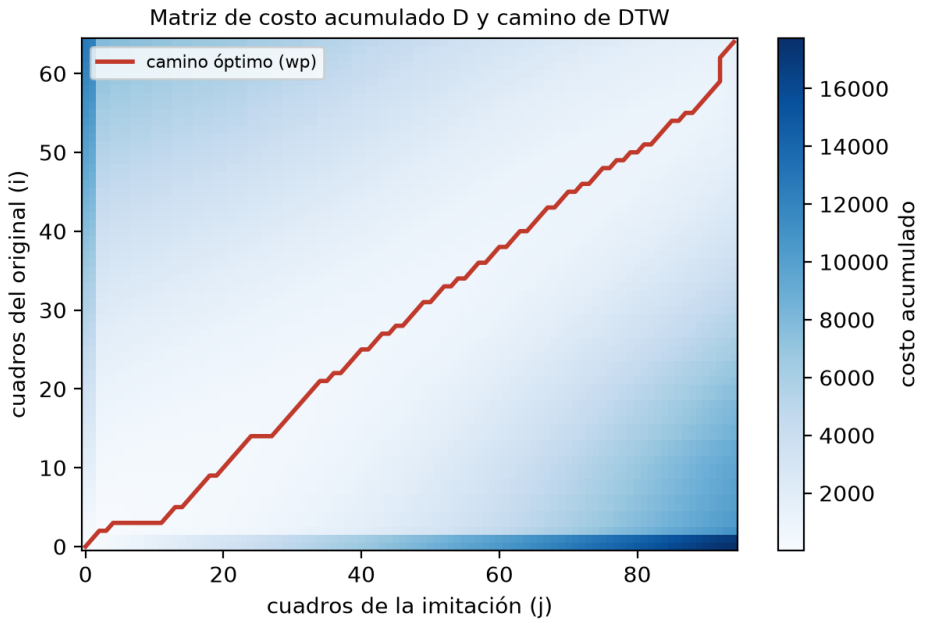


Figura 3. Matriz de costo acumulado entre un silbido original (1,5 s) y una imitación más lenta (2,2 s), con el camino óptimo. Generada con nuestro código.

El costo cuadrático es aceptable para clips cortos (un clip de 3 s tiene unos 130 cuadros, es decir unas 17 000 celdas), pero crece rápido. Por eso `dtw` ofrece una restricción global de banda (`global_constraints=True`, `band_rad`) que pone en infinito las celdas alejadas de la diagonal (`fill_off_diagonal`), y una variante de subsecuencia (`subseq=True`) para buscar un fragmento dentro de una secuencia larga.

pYIN (Mauch & Dixon, 2014). Estima la frecuencia fundamental, es decir el tono, de cada cuadro, y decide si el cuadro tiene tono o no. La Figura 4 resume la implementación de `core/pitch.py`.

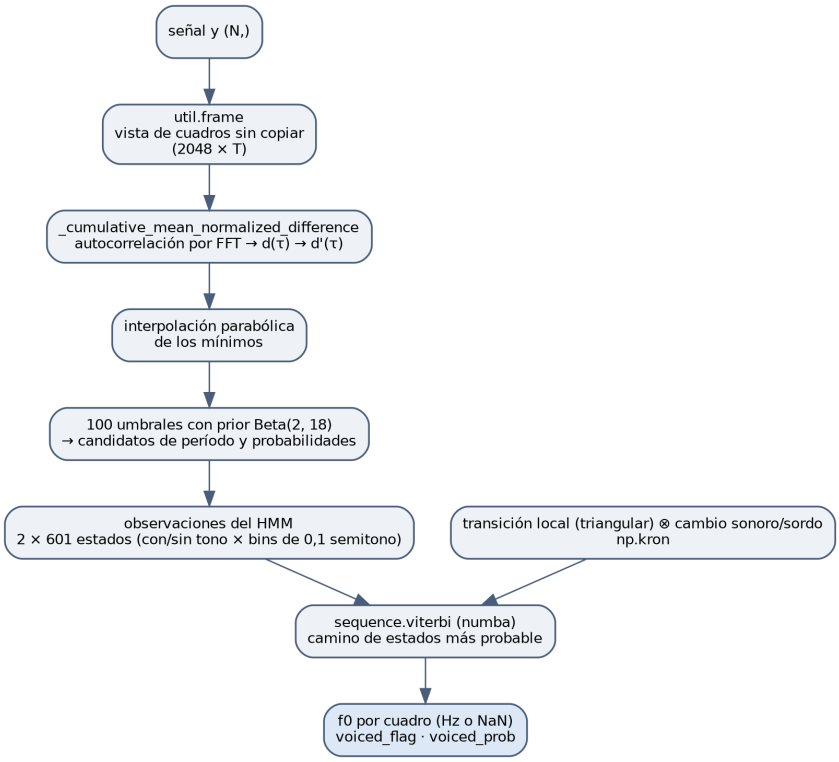


Figura 4. Etapas de librosa.pyin según el código fuente de la versión 1.0.0. Elaboración propia.

1. Parte del algoritmo YIN (de Cheveigné & Kawahara, 2002). Para cada cuadro calcula la función de diferencia d(τ), que mide cuánto se parece la señal a sí misma desplazada τ muestras, y la normaliza por su media acumulada (`_cumulative_mean_normalized_difference`). Los mínimos de esa función son candidatos a período.
2. YIN clásico usa un único umbral fijo para elegir el mínimo. pYIN, en cambio, prueba 100 umbrales ponderados por una distribución Beta(2, 18) y obtiene una **distribución de probabilidad** sobre candidatos. Los candidatos se refinan con interpolación parabólica.
3. Arma un **modelo oculto de Markov** con 2 × B estados: B bandas de tono de 0,1 semitono cada una, con tono y sin tono. Con el rango que usamos (65 Hz a 2093 Hz), B = 601, así que hay 1202 estados. La transición favorece que el tono cambie poco entre cuadros consecutivos.
4. Decodifica la secuencia de estados más probable con **Viterbi** (`sequence._viterbi`, compilado con numba), otro algoritmo de programación dinámica. Su costo completo es $O(T \cdot S^2)$ para S estados. librosa lo reduce a $O(T \cdot S \cdot k)$, donde k es la cantidad de predecesores posibles, porque descarta las transiciones con probabilidad menor que `transition_min_prob` = 10^{-4}.
5. Devuelve tres vectores de largo T: la frecuencia en Hz (NaN en los cuadros sin tono), un indicador booleano de tono y la probabilidad de que haya tono.

La ventaja frente a YIN es doble: la curva sale más estable, porque el HMM penaliza saltos absurdos de octava, y se obtiene una decisión con tono / sin tono, indispensable para no comparar ruido con melodía. La desventaja es el costo. En nuestras mediciones, pYIN tardó unos **190 ms** sobre 3 s de voz, contra unos 7 ms de los MFCC y menos de 1 ms de DTW.

Recorte de silencios (`effects.trim`). Calcula la energía RMS por cuadro, la pasa a decibeles **relativos al máximo de la propia señal** (`ref=np.max`) y conserva desde el primer hasta el último cuadro que supera `−top_db`. Es lineal en el largo de la señal. El detalle de que la referencia es relativa resultó importante en el proyecto (sección 4.14).

Gestión de recursos

- **Memoria.** Se evitan copias con vistas (`util.frame`) y la STFT se procesa en bloques acotados por `MAX_MEM_BLOCK`. Desde la 0.10, `stft` acepta un arreglo de salida preasignado (`out=`) para reutilizar memoria. El tipo por defecto es `float32`, que ocupa la mitad que `float64`. Para archivos muy largos existe `librosa.stream`, un **generador** que entrega la señal por bloques sin cargarla entera.
- **Concurrencia.** librosa no crea hilos propios ni procesos. El paralelismo, cuando lo hay, viene de abajo: las FFT de `scipy.fft` y varias operaciones de NumPy usan código en C con instrucciones SIMD. Para procesar muchos archivos, el usuario puede repartirlos entre procesos con multiprocessing o joblib.
- **Caché.** `_cache.py` define `CacheManager`, un envoltorio de `joblib.Memory`. Las funciones costosas están decoradas con `@cache(level=n)`: `stft` con nivel 20, `filters.mel` con nivel 10. La caché está **apagada por defecto**; se enciende definiendo la variable de entorno `LIBROSA_CACHE_DIR`, y `LIBROSA_CACHE_LEVEL` controla cuánto se guarda. Si está apagada, el decorador devuelve la función original sin envoltorio, con costo cero.
- **E/S.** Solo en `load`, `stream` y funciones relacionadas, siempre a través de soundfile. El archivo se abre con un administrador de contexto (`with`) y se cierra aunque ocurra un error.
- **Errores.** Los parámetros inválidos lanzan `librosa.util.exceptions.ParameterError`, subclase de `LibrosaError`. `util.valid_audio` valida que la señal sea de punto flotante y no contenga NaN ni infinitos antes de procesarla.

Mecanismos de Python que aprovecha

Mecanismo	Dónde y para qué
Carga diferida de módulos	`__init__.py` se reduce a `lazy.attach_stub(__name__, __file__)`: los submódulos se importan recién cuando se usa un nombre, a través de `__getattr__` a nivel de módulo (PEP 562). El archivo `__init__.pyi` declara la interfaz pública. Así `import librosa` es rápido aunque el paquete sea grande
Decoradores	`@cache(level=…)` para la caché; `@jit`, `@stencil` y `@guvectorize` de numba para compilar; `@future_default` y `deprecated` en `util/decorators.py` para avisar cambios de interfaz. El paquete decorator (`FunctionMaker`) preserva la firma original de la función decorada
Compilación JIT con numba	`@jit(nopython=True, cache=True)` traduce funciones Python a código máquina vía LLVM la primera vez que se llaman (Lam et al., 2015). `cache=True` guarda el resultado en `__pycache__` para las siguientes ejecuciones. Se usa en los bucles de DTW y Viterbi, que no se pueden expresar como operaciones vectorizadas
Type hints	Toda la interfaz está anotada. Se usa `typing.overload` para declarar que `dtw` devuelve 1, 2 o 3 valores según sus argumentos, con tipos propios en `_typing.py`. El paquete incluye `py.typed` para que lo verifiquen herramientas como mypy
Argumentos solo por nombre	Las firmas usan `*`, como en `load(path, *, sr=22050, …)` o `mfcc(*, y=None, sr=22050, …)`. Escribir `librosa.load(ruta, 16000)` da error: hay que escribir `sr=16000`. Evita confusiones entre parámetros numéricos
Generadores	`librosa.stream` produce bloques con `yield` para procesar audio largo
Extensiones en C	Indirectamente: NumPy y SciPy están escritas en C/C++/Fortran, soundfile y soxr envuelven bibliotecas en C. librosa en sí es Python puro (su wheel es `py3-none-any`)

Tabla 4. Mecanismos de Python usados por librosa.

Decisiones de diseño

- **Funciones en lugar de clases.** Las funciones sin estado se componen fácilmente, se prueban por separado y se integran con el resto del ecosistema, que también trabaja con `ndarray`. La alternativa, objetos "Audio" con métodos, ataría al usuario a tipos propios.
- **Valores por defecto conservadores.** `load` convierte a mono y a 22 050 Hz salvo que se indique otra cosa. La mitad de la frecuencia de CD alcanza para la mayoría de los análisis musicales y reduce a la mitad el costo de todo lo que sigue.
- **Delegar lo especializado.** FFT en SciPy, lectura de archivos en libsndfile, remuestreo en soxr. La 1.0 eliminó audioread y simplificó la carga a un solo camino.
- **numba en lugar de extensiones en C.** Mantiene el paquete en Python puro, sin compilación al instalar y con un código fuente legible, a cambio de un tiempo de compilación en el primer uso.
- **Estabilidad.** Los cambios de interfaz se anuncian con avisos de obsolescencia durante al menos una versión antes de concretarse. La 1.0 "venció" esos avisos y removió lo marcado.

4.4. ¿Para qué se usa?

librosa resuelve problemas de **análisis** de audio, es decir, extraer información de un sonido grabado. Sus dominios de aplicación son:

- Investigación en MIR: detección de tempo y pulsos, reconocimiento de acordes, transcripción, segmentación de canciones y búsqueda por similitud.
- Preparación de datos para aprendizaje automático: espectrogramas mel y MFCC como entrada de redes neuronales de clasificación de audio, reconocimiento de voz o síntesis.
- Análisis de voz y fonética: curvas de tono, energía y duración.
- Docencia: es la biblioteca de referencia de libros y cursos de procesamiento de música (Müller, 2015).

Cuándo conviene y cuándo no

Conviene si...	No conviene si...
Se procesan archivos ya grabados o clips cortos (análisis fuera de línea)	Se necesita procesar en tiempo real con baja latencia, por ejemplo un afinador o efectos en vivo: librosa trabaja sobre el arreglo completo y la primera llamada compila
Se quiere prototipar rápido y leer el código del algoritmo	Se necesita máximo rendimiento en lote sobre miles de horas de audio o cálculo en GPU (ahí rinden más torchaudio o Essentia)
Se trabaja dentro del ecosistema NumPy/SciPy/scikit-learn	Se necesita sintetizar, mezclar o editar audio para producción: librosa analiza, no es un editor
Se necesitan algoritmos clásicos de MIR documentados con su paper	El entorno no admite dependencias pesadas (dispositivos embebidos) o usa Python < 3.12

Tabla 5. Casos en que librosa conviene y casos en que no.

Ventajas y desventajas

- **Ventajas:** interfaz uniforme y bien documentada, con ejemplos y referencias al paper de cada método. Código fuente legible en Python. Algoritmos completos, como pYIN con HMM o DTW con restricciones, que serían largos de implementar. Licencia permisiva. Gran comunidad, por lo que es fácil encontrar respuestas.
- **Desventajas:** dependencias pesadas, ya que la instalación arrastra NumPy, SciPy, numba con LLVM y scikit-learn, cientos de MB en total. Latencia de la primera llamada por la compilación de numba, que en nuestra prueba fue de unos 13 s la primera vez y 1,6 s las siguientes. Solo CPU. Algunas funciones tienen detalles que hay que conocer para usarlas bien, como la referencia relativa de `trim`.

4.5. Casos de uso reales

librosa es sobre todo una herramienta de investigación y de preparación de datos, así que la mayoría de sus usos documentados son proyectos de código abierto y trabajos académicos, más que casos corporativos con métricas publicadas. Los casos que pudimos verificar en fuentes primarias son:

- **OpenAI Whisper** (reconocimiento de voz, Radford et al., 2023). El archivo `whisper/audio.py` del repositorio oficial calcula el espectrograma log-mel que recibe el modelo con un banco de filtros generado con `librosa.filters.mel(sr=16000, n_fft=400, n_mels=80)`. Según el comentario del propio código, la matriz se guardó en `mel_filters.npz` para no depender de librosa en tiempo de ejecución. Es un ejemplo de librosa usada como **referencia de corrección**: se toma su implementación como la versión correcta del banco de filtros mel.
- **Herramientas derivadas de investigación en voz**, como el toolkit Amphion de conversión de voz cantada, que reutilizan ese extractor basado en Whisper y librosa (publicado en Hugging Face Spaces).
- **Investigación académica.** El artículo de presentación (McFee et al., 2015) es una de las referencias más citadas de la conferencia SciPy, y cada versión tiene un DOI propio en Zenodo para que los trabajos citen exactamente la versión que usaron.

No encontramos casos de empresas que publiquen resultados cuantitativos obtenidos con librosa. Lo indicamos explícitamente, como pide la consigna. Antes de la entrega conviene verificar los enlaces de la sección de referencias, porque los repositorios cambian.

4.6. Comparación con alternativas

Criterio	librosa	Essentia	aubio	torchaudio
Lenguaje del núcleo	Python puro sobre NumPy/SciPy + numba	C++ con enlaces a Python	C con enlaces a Python	C++/CUDA sobre PyTorch
Rendimiento	Medio; primera llamada lenta (JIT)	Alto	Alto, pensado para tiempo real	Alto; aprovecha GPU
Modelo de ejecución	Funciones sobre arreglos completos	Algoritmos encadenados en redes de flujo (modo estándar o *streaming*)	Procesamiento por bloques con estado	Operaciones sobre tensores, diferenciables
Curva de aprendizaje	Baja; documentación con ejemplos	Media-alta	Media	Media; requiere conocer PyTorch
Ecosistema / madurez	Muy amplio en investigación; 1.0 estable	Maduro (MTG, Universitat Pompeu Fabra)	Maduro, comunidad más chica	Respaldado por el ecosistema PyTorch
Licencia	ISC (permisiva)	AGPL-3.0	GPL-3.0	BSD-2
Caso ideal	Análisis y prototipos de MIR	Extracción masiva y productos	Afinadores, detección en vivo	Entrenar redes neuronales con audio

Tabla 6. Comparación de librosa con alternativas. Fuentes: Bogdanov et al. (2013), Brossier (2006), Yang et al. (2022) y documentación oficial.

Por qué elegimos librosa. El proyecto analiza clips cortos ya grabados, así que no necesita tiempo real (la ventaja de aubio) ni GPU (la de torchaudio). Necesita tres algoritmos concretos, MFCC, un estimador de tono con decisión con tono / sin tono y DTW, y librosa es la única de

las cuatro que los trae todos con la misma interfaz. Además, para este trabajo pesó mucho que el código fuente esté en Python legible: pudimos leer cómo funciona cada algoritmo, cosa más difícil en las alternativas en C o C++. Finalmente, su licencia permisiva no impone condiciones a nuestro código, a diferencia de la AGPL de Essentia o la GPL de aubio.

4.7. Instalación, ecosistema y buenas prácticas

Requisitos e instalación

librosa 1.0.0 requiere Python 3.12 o superior y se instala con `pip install librosa` o, con conda, `conda install -c conda-forge librosa`. En Windows, macOS y Linux las dependencias se descargan precompiladas (*wheels*), así que no hace falta compilador. La instalación funciona dentro de un entorno virtual como cualquier paquete. `librosa.show_versions()` muestra las versiones de todas las dependencias, útil para reportar problemas.

Herramientas complementarias

- **librosa.display** (requiere matplotlib, extra `display`): grafica formas de onda y espectrogramas con ejes en segundos, Hz o notas.
- **sounddevice** y **soundfile**: grabar y reproducir, y leer y escribir archivos. librosa no graba del micrófono.
- **Otros motores de FFT**: la 1.0 llama directamente a `scipy.fft`, así que se puede cambiar la implementación con `scipy.fft.set_backend`, por ejemplo con la interfaz de pyFFTW.
- **mir_eval**, **jams** y **muda**, del mismo grupo de autores: evaluación de algoritmos MIR, formato de anotaciones y aumento de datos.

Buenas prácticas y errores comunes

- **Fijar la frecuencia de muestreo.** Si se comparan dos audios, cargarlos con el mismo `sr`; si no, los cuadros no representan la misma duración.
- **Usar el mismo** `hop_length` en todas las características que se van a combinar, para que el cuadro i signifique el mismo instante en todas.
- **No olvidar los argumentos por nombre**: `librosa.feature.mfcc(y=y, sr=sr)`, no `mfcc(y, sr)`.
- **Manejar los NaN de pyin.** Los cuadros sin tono valen NaN y cualquier promedio que los incluya da NaN.
- **Recordar que** `trim` **es relativo.** No detecta una grabación que es solo ruido.
- **Precalentar** las funciones compiladas con numba antes de un momento crítico, o aceptar la demora de la primera llamada.
- **Activar la caché** (`LIBROSA_CACHE_DIR`) en experimentos que recalculan las mismas características muchas veces.
- **Citar la versión** exacta usada (`librosa.cite()`) en trabajos académicos.

PARTE II — El proyecto: Voicer Choicer en consola

4.8. Descripción introductoria

Voicer Choicer es un juego de imitación de voz: el juego hace sonar un audio corto y los participantes intentan reproducirlo con su voz lo más parecido posible. Nuestro proyecto es una versión para consola. Cubre una necesidad lúdica, un juego de fiesta para jugar en una misma computadora, y a la vez resuelve un problema técnico concreto: **medir automáticamente cuánto se parecen dos sonidos** aunque los produzcan personas distintas, a distinta velocidad y con distinta altura de voz.

Alcance

Incluye	No incluye
De 1 a 3 jugadores por turnos en la misma computadora y el mismo micrófono	Juego en red o desde varias computadoras
Packs de clips como carpetas de archivos .wav, más un pack demo generado por script	Descarga de clips de internet o packs con derechos de autor
Grabación de clips propios desde el menú	Edición de audio (recortar, filtrar)
Puntaje de 0 a 100 por timbre y por tono	Reconocimiento de palabras (qué se dijo)
Ranking persistente en un archivo JSON	Base de datos, cuentas de usuario o servidor
Interfaz de texto con menús, barras y tablas	Interfaz gráfica o web

Tabla 7. Alcance del proyecto.

Requisitos funcionales principales

1. Mostrar un menú con las opciones jugar, ver ranking, grabar un clip nuevo y salir.
2. Permitir elegir un pack, la cantidad de jugadores (1 a 3) y sus nombres, validando que no estén vacíos ni repetidos.
3. En cada ronda, elegir al azar un clip no repetido, reproducirlo y grabar la imitación de cada jugador con cuenta regresiva.
4. Calcular y mostrar el puntaje de timbre, de tono y total de cada imitación.
5. Detectar grabaciones vacías y dar un reintento.
6. Al final, mostrar el ganador y guardar el resultado en el ranking.
7. Mostrar las 10 mejores partidas ordenadas por promedio por ronda.
8. Grabar un clip nuevo y guardarlo en un pack existente o nuevo sin pisar archivos.

Rol de la tecnología investigada

librosa resuelve el corazón del sistema: el módulo `puntaje.py`, que convierte dos audios en un número. Concretamente, se usan seis de sus funciones:

- `librosa.load`: lee los clips y los lleva a 22 050 Hz mono.

- `effects.trim` y `util.normalize`: quitan el silencio de los extremos y la diferencia de volumen.
- `feature.mfcc`: describe el timbre.
- `pyin`: describe la melodía.
- `sequence.dtw`: alinea en el tiempo original e imitación.

Era una buena elección porque estas tres técnicas, MFCC, estimación de tono y DTW, son justamente las clásicas para comparar voz (Müller, 2015). Implementarlas a mano habría llevado gran parte del tiempo del trabajo, en especial pYIN con su modelo oculto de Markov. Sin librosa, el proyecto se reduciría a comparar volúmenes. Con librosa es un juego que efectivamente distingue una buena imitación de una mala.

4.9. Estructura del proyecto

```
voicer-choicer/
├── README.md               instalación, ejecución y uso rápido
├── pyproject.toml          metadatos del paquete, dependencias, config. de ruff y black
├── requirements.txt        dependencias directas con versión fija
├── .gitignore              excluye .venv/, __pycache__/, datos/ y grabaciones/
├── src/
│   └── voicer_choicer/     paquete del juego
│       ├── __init__.py     versión del paquete
│       ├── __main__.py     permite "python -m voicer_choicer"
│       ├── main.py         punto de entrada: logging, menú y manejo de errores
│       ├── juego.py        clase Partida: rondas, turnos y puntos
│       ├── interfaz.py     toda la entrada/salida de consola
│       ├── puntaje.py      análisis y comparación (único módulo que usa librosa)
│       ├── audio.py        micrófono, parlantes y .wav (sounddevice, soundfile)
│       ├── packs.py        búsqueda y elección de clips en carpetas
│       ├── ranking.py      lectura y escritura del ranking en JSON
│       ├── config.py       todas las constantes ajustables
│       └── errores.py      jerarquía de excepciones propias
├── tests/                  24 pruebas con unittest
│   ├── test_puntaje.py     análisis y comparación con señales sintéticas
│   ├── test_partida.py     partida completa con audio simulado (mocks)
│   ├── test_packs.py       manejo de carpetas y nombres
│   └── test_ranking.py     persistencia, archivo dañado, orden
├── packs/
│   └── demo/               5 clips sintéticos (silbidos y notas)
├── herramientas/
│   └── generar_pack_demo.py   genera packs/demo con librosa.chirp y librosa.tone
├── docs/                   este informe y los diagramas
├── datos/                  (se crea al jugar) ranking.json y voicer.log
└── grabaciones/            (se crea al jugar) cada imitación como .wav
```

Justificación de la estructura

Consideramos tres formas de organizar el código. La primera era un único archivo, la más simple, pero con unas 800 líneas mezclaría la consola, el micrófono y el análisis, y no se podría probar nada por separado. La segunda era una organización por capas al estilo del ejemplo de la consigna (carpetas `models/`, `services/`, `infra/`). Es razonable para sistemas grandes, pero para nueve módulos agregaría carpetas con uno o dos archivos cada una. Elegimos un **monolito modular plano**: un paquete con un módulo por responsabilidad. Esa separación conserva lo esencial de las capas sin la ceremonia:

- **Presentación**: `main.py` e `interfaz.py`.
- **Lógica del juego**: `juego.py`.

- **Dominio del análisis**: `puntaje.py`.
- **Infraestructura**: `audio.py`, `packs.py` y `ranking.py`.

Los criterios que aplicamos fueron:

- **Cohesión.** Cada módulo tiene una sola razón para cambiar. Si se recalibra el puntaje se toca `config.py` o `puntaje.py`; si se cambia de biblioteca de micrófono, solo `audio.py`.
- **Bajo acoplamiento con librosa.** Solo `puntaje.py` la importa. El resto del programa recibe `float` y `list` de Python, sin saber que existen los arreglos de NumPy. Esto cumple además una restricción que nos pusimos: no usar NumPy directamente en nuestro código.
- **Testeabilidad.** Como la interfaz y el audio están aislados, las pruebas los reemplazan por *mocks* y ejercitan la lógica real sin micrófono ni teclado.
- **Diseño** `src/`**.** El código vive en `src/voicer_choicer/` y se instala con `pip install -e .`. Así las pruebas importan el paquete instalado, igual que el usuario, y no funcionan "de casualidad" por estar en la carpeta correcta.

Convenciones adoptadas

- **Idioma:** identificadores, comentarios y mensajes en español, sin ñ en los nombres (`TAMANIO_VENTANA`), porque Python lo permite pero complica escribirlos en algunos teclados.
- **Nombres:** `snake_case` para funciones y variables, `PascalCase` para clases, `MAYÚSCULAS` para constantes y guion bajo inicial para lo interno (`_alinear`).
- **Estilo:** formateado con black y verificado con ruff (sin avisos), con largo de línea 88 y anotaciones de tipo en todas las funciones.
- **Configuración:** sin números mágicos; todo valor ajustable está en `config.py`, con un comentario que explica su unidad y su efecto. No hay credenciales ni datos sensibles.
- **Rutas:** siempre con `pathlib.Path` relativas a la raíz del proyecto, para que funcionen igual en Windows y en Linux.

4.10. Cómo ejecutar el proyecto

El README del proyecto tiene estas mismas instrucciones, con una tabla de problemas frecuentes. Las resumimos para Windows, el sistema que usa el grupo.

Requisitos previos

- Windows 10 u 11 (también funciona en Linux y macOS).
- Python 3.12 o 3.13 desde python.org, con la opción "Add python.exe to PATH". librosa 1.0 no se instala en versiones anteriores.
- Micrófono y auriculares. Los auriculares evitan que el micrófono capte el clip original.
- Internet solo para instalar dependencias. No hay servicios externos ni variables de entorno obligatorias.

Pasos

```
REM 1. Ubicarse en la carpeta del proyecto (la que tiene README.md)
cd voicer-choicer

REM 2. Crear y activar el entorno virtual
py -3.12 -m venv .venv
.venv\Scripts\activate
REM    Si PowerShell bloquea el script, una sola vez:
REM    Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

REM 3. Instalar dependencias y el paquete
python -m pip install --upgrade pip
pip install -r requirements.txt
pip install -e .

REM 4. Ejecutar el juego
python -m voicer_choicer

REM 5. Correr las pruebas
python -m unittest discover -s tests -v
```

Configuración: no se necesita ninguna. Las carpetas `datos/` y `grabaciones/` se crean solas. El pack `demo` ya viene incluido y se puede regenerar con `python herramientas/generar_pack_demo.py`. Opcionalmente, todos los parámetros del juego y del puntaje se ajustan en `config.py`.

Problemas frecuentes

Síntoma	Causa y solución
`requires a different Python` al instalar	Python < 3.12. Instalar 3.12 o 3.13 y recrear el .venv con `py -3.12`
"No se encontró ningún micrófono conectado"	Conectar el micrófono antes de abrir el juego y elegirlo como entrada predeterminada en Configuración → Sistema → Sonido
Siempre dice "No se escuchó nada"	Permiso de micrófono desactivado para apps de escritorio (Configuración → Privacidad y seguridad → Micrófono) o volumen de entrada muy bajo
La primera ronda tarda varios segundos	Compilación de numba; pasa una vez y luego queda en caché
`ModuleNotFoundError: voicer_choicer`	Falta activar el entorno o correr `pip install -e .`

Tabla 8. Problemas frecuentes al ejecutar.

4.11. Instrucciones de uso

Flujo principal: jugar una partida

1. Ejecutar `python -m voicer_choicer`. El programa muestra el micrófono detectado y prepara el motor de audio.
2. Elegir **1. Jugar**, el pack (por ejemplo `demo`), la cantidad de jugadores y sus nombres.
3. En cada una de las 3 rondas, cada jugador presiona Enter para escuchar el clip, Enter cuando está listo, y después de la cuenta regresiva 3, 2, 1 imita el sonido. La grabación dura lo mismo que el clip más un segundo de margen.

4. Se muestran las barras de timbre, tono y total. Al terminar la ronda se ve el acumulado.
5. Al final aparece el podio, y el resultado queda guardado en el ranking.

Ejemplo real de salida, generado con un micrófono simulado que "imita" reproduciendo el mismo clip:

```
================================================
                  RONDA 1 DE 3
================================================
Clip: Silbido que baja

Turno de Ana. Escuchá el original:
Enter para escuchar...
Enter cuando estés listo para imitarlo...
  3...
  2...
  1...
  ¡AHORA!
  Analizando...

  Ana
    Timbre  ████████████████▒▒  92
    Tono    ████████████████▒▒  94
    TOTAL   ████████████████▒▒  93
```

Validación de entradas

```
¿Cuántos jugadores? [1-3]: 5
[!] Tiene que estar entre 1 y 3.
¿Cuántos jugadores? [1-3]: dos
[!] Escribí un número.
¿Cuántos jugadores? [1-3]: 2
Nombre del jugador 1: Ana
Nombre del jugador 2: ana
[!] Ese nombre ya está en uso.
```

Ver ranking

```
================================================
           RANKING (PROMEDIO POR RONDA)
================================================
#  Jugador  Promedio  Pack  Fecha
------------------------------------------
1  Ivan     95.7      demo  2026-10-27 16:50
```

Grabar un clip nuevo

La opción **3** pide un nombre de pack (existente o nuevo) y un nombre de clip, cuenta 3, 2, 1, graba 4 segundos y guarda el archivo, por ejemplo `packs/voces/risa_malvada.wav`. Si ya existe un clip con ese nombre, lo guarda como `risa_malvada_2.wav`. Si la grabación quedó en silencio, no se guarda. Los nombres se limpian de caracteres que Windows no acepta, como `: ? *`.

Cómo leer el puntaje. Timbre alto significa que "suena" parecido: mismas vocales, misma aspereza, mismos ruidos. Tono alto significa que la melodía sube y baja en los mismos lugares, aunque sea más grave o más aguda que el original. El total pondera 60 % timbre y 40 % tono. En clips sin melodía, como un portazo, se muestra "(el clip no tiene melodía)" y cuenta solo el timbre.

4.12. Descripción estructural en profundidad

Arquitectura general

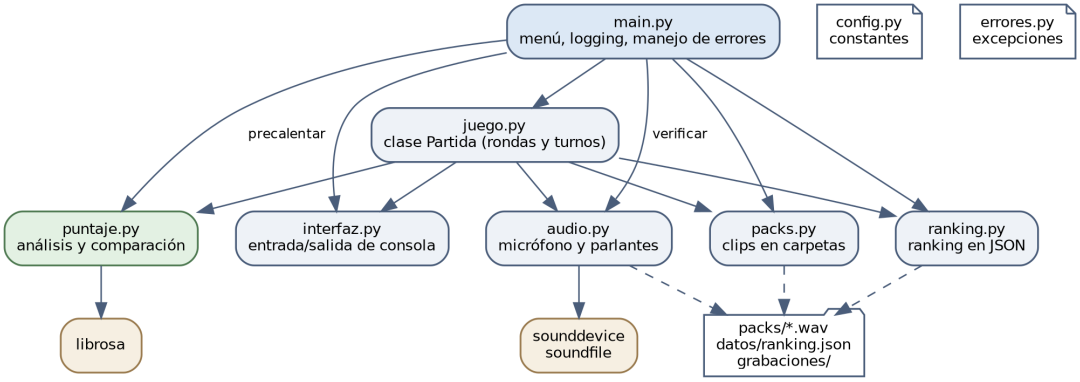


Figura 5. Módulos del proyecto y sus dependencias. El módulo de análisis (verde) es el único que depende de librosa. Elaboración propia.

Las flechas apuntan siempre "hacia abajo": la presentación usa la lógica, la lógica usa el análisis y la infraestructura, y nada de abajo conoce lo de arriba. `config.py` y `errores.py` son importados por casi todos y no importan a nadie del proyecto, de modo que no puede haber ciclos de importación.

Módulos y clases principales

Módulo	Responsabilidad	Elementos clave
main.py	Arranque, menú principal, captura de errores esperables	`main()`, `menu_jugar()`, `menu_ranking()`, `menu_grabar_clip()`, `configurar_logging()`
juego.py	Una partida: rondas, turnos, acumulación	Clase `Partida`
puntaje.py	Convertir audio en descripción y comparar	`Analisis`, `Resultado`, `analizar()`, `comparar()`, `precalentar()`
audio.py	Micrófono, parlantes y .wav	`verificar_microfono()`, `reproducir()`, `grabar()`, `guardar()`, `duracion()`
packs.py	Encontrar y elegir clips	`listar_packs()`, `listar_clips()`, `elegir_clips()`, `siguiente_nombre_libre()`
ranking.py	Persistir y ordenar resultados	`cargar()`, `guardar()`, `registrar_partida()`, `mejores()`
interfaz.py	Toda la E/S de consola	`pedir_entero()`, `pedir_texto()`, `barra()`, `mostrar_tabla()`
config.py / errores.py	Constantes / excepciones	`ErrorJuego` y sus subclases

Tabla 9. Responsabilidad de cada módulo.

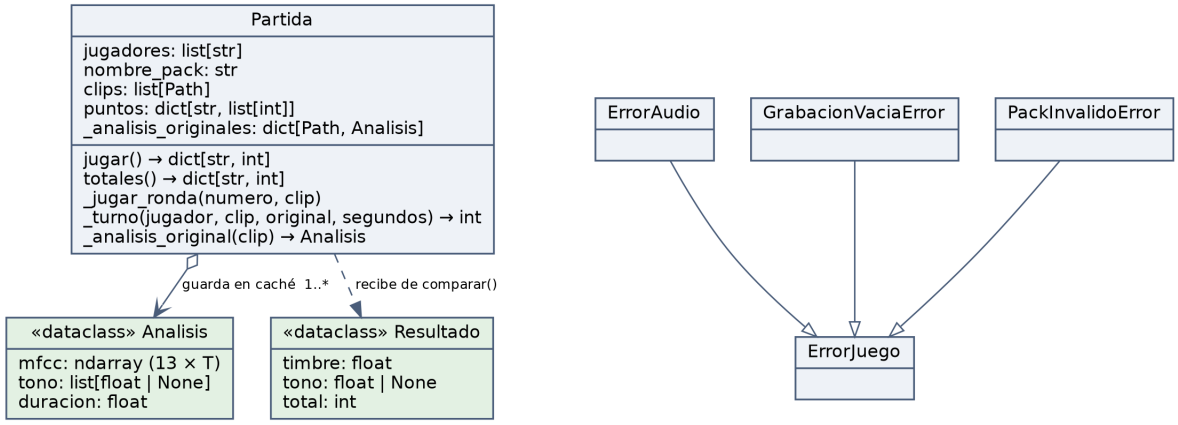


Figura 6. Diagrama de clases: Partida, las dataclasses del análisis y la jerarquía de excepciones. Elaboración propia.

`Analisis` y `Resultado` son `@dataclass`: registros con nombre en lugar de tuplas, por lo que `resultado.timbre` se lee mejor que `resultado[0]` y el método `__repr__` generado sirve para el log. `Partida` es la única clase con estado propio: los jugadores, los clips de la partida, los puntos y una caché de análisis. Las excepciones forman una jerarquía con raíz `ErrorJuego`, así `main.py` captura cualquier error "esperable" con un solo `except` sin tapar errores de programación como `TypeError`.

Ciclo de ejecución

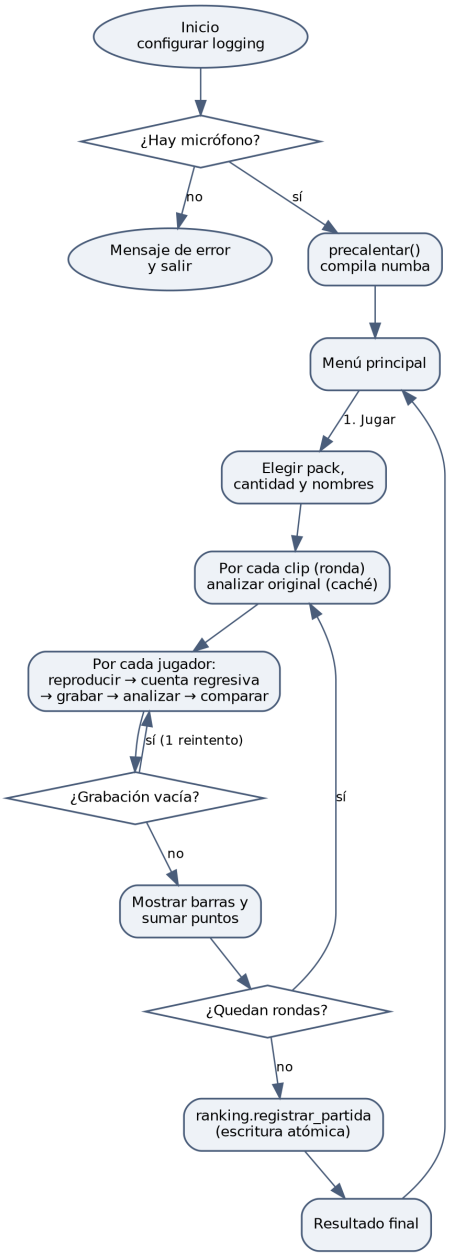


Figura 7. Ciclo de ejecución del programa, desde el arranque hasta el fin de una partida. Elaboración propia.

Al arrancar, `main()` configura el logging, verifica que haya micrófono (si no lo hay, termina con un mensaje claro) y llama a `puntaje.precalentar()`, que analiza un tono de 1 s para forzar la compilación de numba. Luego entra en el bucle del menú, del que solo sale con la opción 4 o si se cierra la entrada estándar. La Figura 8 detalla un turno, la operación central.

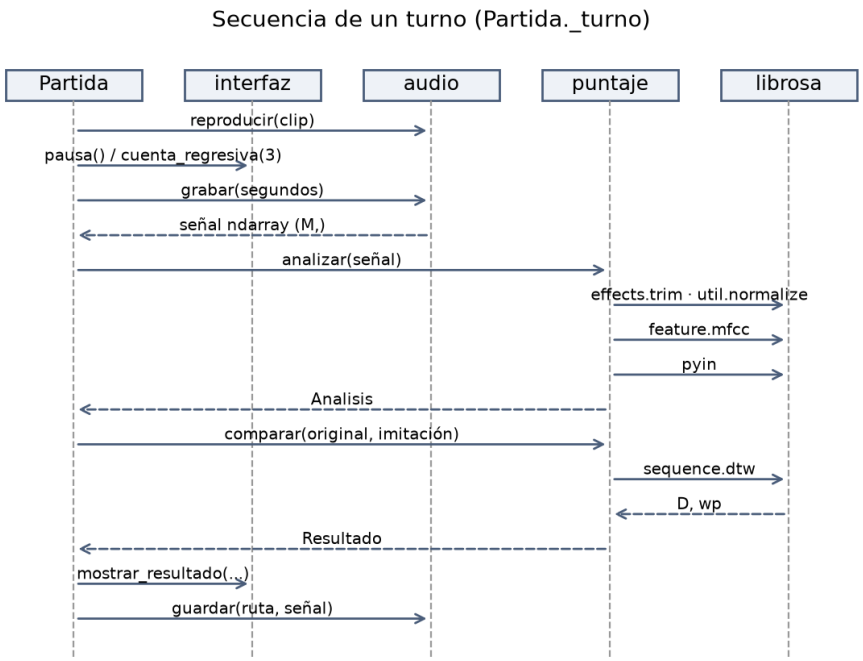


Figura 8. Diagrama de secuencia de un turno. Las flechas punteadas son valores devueltos. Elaboración propia.

Flujo de datos

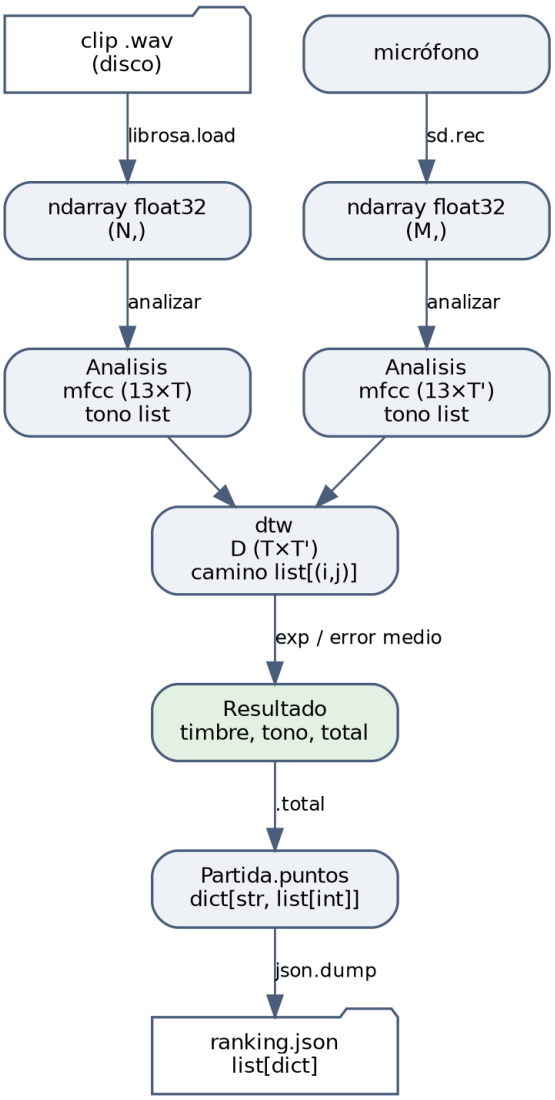


Figura 9. Transformación de los datos entre componentes. Elaboración propia.

Un clip entra como archivo .wav y la imitación como búfer del micrófono. Ambos se convierten en un `ndarray` de `float32`, y `analizar()` los reduce a un `Analisis`. Un clip de 2 s pasa de 44 100 muestras a una matriz de 13 × 87 y una lista de 87 tonos, unas 50 veces menos datos. La comparación produce un camino de DTW y de ahí un `Resultado` con tres números. El total se acumula en un diccionario por jugador y al final se escribe como lista de diccionarios en JSON.

Estructuras de datos utilizadas en el proyecto

Estructura	Dónde	Por qué y complejidad
`dict[str, list[int]]`	`Partida.puntos`	Acceso a los puntos de un jugador por nombre en O(1) promedio (tabla hash). La lista de cada jugador guarda una entrada por ronda, con append O(1) amortizado
`dict[Path, Analisis]`	`Partida._analisis_originales`	Caché (memoización): cada clip original se analiza una vez aunque lo imiten 3 jugadores. Ahorra 2 de 3 llamadas a pyin por ronda, unos 190 ms cada una. Búsqueda O(1)
`list[Path]`	`Partida.clips`	Orden de las rondas; se recorre una vez, O(n)
`random.sample`	`packs.elegir_clips`	Elige k clips sin repetición en O(k), sin mezclar toda la lista
`list[tuple[int, int]]`	camino de DTW	Se recorre secuencialmente una sola vez para comparar tonos, O(N+M). Se convierte de ndarray a tuplas nativas para que el resto del código no dependa de NumPy
`list[float | None]`	`Analisis.tono`	Curva de tono con `None` en los cuadros sin tono. Es más explícito que NaN, que se propaga en silencio en cualquier cuenta
ndarray (13 × T)	`Analisis.mfcc`	La única estructura de NumPy que conservamos, porque se le pasa directo a `librosa.sequence.dtw`. Convertirla a listas y volver costaría O(13·T) dos veces sin beneficio
`set[str]`	`interfaz.pedir_texto`	Nombres ya usados; la pertenencia (`in`) es O(1)
`list[dict]` en JSON	`ranking.json`	Orden de llegada; se ordena al consultar con `sorted` (Timsort, O(n log n)) por promedio por ronda. Para un ranking de cientos de partidas es instantáneo

Tabla 10. Estructuras de datos del proyecto.

Puntos de integración con la tecnología investigada

Toda la integración está en `puntaje.py`. A continuación, la función `analizar` tal como está en el código (sin el *docstring*):

```
def analizar(senal, sr: int = config.FRECUENCIA_MUESTREO) -> Analisis:
    # 0. librosa.effects.trim mide el silencio en forma RELATIVA al pico de la
    #    propia señal, así que una grabación que es solo ruido de fondo no se
    #    detecta como vacía. Por eso primero se mira el volumen absoluto.
    pico = float(abs(senal).max()) if len(senal) else 0.0
    if pico < config.PICO_MINIMO:
        raise GrabacionVaciaError(
            "No se escuchó nada. Acercate al micrófono y probá de nuevo."
        )

    # 1. Recortar silencios al principio y al final. Sin esto, el medio
    #    segundo que tarda el jugador en arrancar empeora el puntaje.
    recortada, _ = librosa.effects.trim(senal, top_db=config.UMBRAL_SILENCIO_DB)
    duracion = len(recortada) / sr
    if duracion < config.DURACION_MINIMA_SEG:
        raise GrabacionVaciaError(
            "No se escuchó nada. Acercate al micrófono y probá de nuevo."
        )

    # 2. Normalizar para que gritar o susurrar no cambie el resultado.
    recortada = librosa.util.normalize(recortada)

    # 3. Timbre: MFCC. Se descarta el coeficiente 0 (es casi la energía
```

```
    #    total, o sea el volumen) y se resta la media de cada coeficiente
    #    (normalización cepstral), que compensa diferencias de micrófono.
    mfcc = librosa.feature.mfcc(
        y=recortada,
        sr=sr,
        n_mfcc=config.CANTIDAD_MFCC,
        n_fft=config.TAMANIO_VENTANA,
        hop_length=config.SALTO,
    )[1:]
    mfcc = mfcc - mfcc.mean(axis=1, keepdims=True)

    # 4. Tono: pyin devuelve la frecuencia fundamental por cuadro (NaN si
    #    el cuadro no tiene tono). Usa el mismo SALTO que los MFCC, así el
    #    cuadro i de una curva corresponde al cuadro i de la otra.
    f0, _, _ = librosa.pyin(
        recortada,
        fmin=librosa.note_to_hz(config.NOTA_MINIMA),
        fmax=librosa.note_to_hz(config.NOTA_MAXIMA),
        sr=sr,
        frame_length=config.TAMANIO_VENTANA,
        hop_length=config.SALTO,
    )
    tono = _a_semitonos(f0.tolist())

    return Analisis(mfcc=mfcc, tono=tono, duracion=duracion)
```

Varias decisiones de este fragmento salen directamente de lo investigado en la Parte I:

- **Mismo** `hop_length` **(512) en MFCC y pyin.** Ambas funciones centran los cuadros de la misma forma, así que producen la misma cantidad de cuadros y el cuadro i de una corresponde al cuadro i de la otra. Gracias a eso, el camino de DTW calculado sobre los MFCC sirve también para comparar los tonos.
- **Se descarta el coeficiente 0 de los MFCC.** Por cómo funciona la DCT, el primer coeficiente es proporcional al promedio del espectro en decibeles, es decir, al volumen. Lo quitamos para que hablar más fuerte no cambie el timbre.
- **Se resta la media de cada coeficiente** (normalización cepstral). Un micrófono distinto multiplica el espectro por una curva fija; en el dominio logarítmico eso es una suma, y tras la DCT queda como una constante en cada coeficiente. Restar la media la elimina.
- **pyin con** `fmin` **C2 y** `fmax` **C7.** Cubre desde voces graves hasta silbidos. El rango determina la cantidad de estados del HMM (1202), y con ella el costo de Viterbi.
- `f0.tolist()`**.** Convierte el arreglo a `float` de Python, y los NaN se pasan a `None` en `_a_semitonos`.

La alineación usa `librosa.sequence.dtw`. Como el costo total D[N, M] crece con la cantidad de pasos, lo dividimos por el largo del camino para obtener un **costo medio por paso**, comparable entre clips cortos y largos:

```
def _alinear(
    original: Analisis, imitacion: Analisis
) -> tuple[list[tuple[int, int]], float]:
    costo_acumulado, camino = librosa.sequence.dtw(
        X=original.mfcc, Y=imitacion.mfcc, metric="euclidean"
    )
    pares = [(int(i), int(j)) for i, j in camino.tolist()]
    costo_medio = float(costo_acumulado[-1, -1]) / len(pares)
    return pares, costo_medio
```

Para el tono, sobre ese mismo camino se calcula la **mediana de las diferencias** entre las dos curvas, que representa cuántos semitonos más grave o más aguda cantó el jugador, y se la

descuenta. Además, los pasos donde el original tiene tono y la imitación no cuentan como error máximo:

```
def _puntaje_tono(
    tono_original: list[float | None],
    tono_imitacion: list[float | None],
    camino: list[tuple[int, int]],
) -> float | None:
    tope = config.ERROR_TONO_MAXIMO
    pares = [
        (tono_original[i], tono_imitacion[j])
        for i, j in camino
        if i < len(tono_original)
        and j < len(tono_imitacion)
        and tono_original[i] is not None
    ]
    if len(pares) < config.PARES_SONOROS_MINIMOS:
        return None  # el original casi no tiene tono: no se evalúa

    diferencias = [a - b for a, b in pares if b is not None]
    if len(diferencias) < config.PARES_SONOROS_MINIMOS:
        return 0.0  # el original tenía melodía y la imitación no
    corrimiento = statistics.median(diferencias)

    errores = [
        tope if b is None else min(abs(a - b - corrimiento), tope) for a, b in pares
    ]
    return 100.0 * (1.0 - statistics.fmean(errores) / tope)
```

Manejo de errores, configuración y logging

- **Errores esperables** → excepciones propias con mensaje para el usuario: `ErrorAudio` (sin micrófono, archivo dañado), `GrabacionVaciaError` (silencio, con un reintento por turno) y `PackInvalidoError`. `main.py` las captura, las informa y vuelve al menú. Ctrl+C cancela la acción en curso sin cerrar el programa.
- **Encapsular errores de bibliotecas.** `audio.py` traduce `sd.PortAudioError` y el `RuntimeError` de soundfile a `ErrorAudio`, con `raise … from error` para conservar la causa original. Si PortAudio ni siquiera está instalado, la importación de sounddevice lanza `OSError`. Se captura al importar y se informa recién al intentar usar el audio, lo que permite correr las pruebas en máquinas sin placa de sonido.
- **Datos dañados.** Si `ranking.json` no es JSON válido, se renombra a `ranking.corrupto`, para no perderlo, y se empieza uno nuevo. Las entradas con forma incorrecta se ignoran.
- **Escritura atómica.** El ranking se escribe en un archivo temporal y se reemplaza con `os.replace`, que es atómico en el mismo disco. Si el programa se corta a mitad de escritura, el ranking anterior queda intacto:

```
def guardar(entradas: list[dict], archivo: Path = config.ARCHIVO_RANKING) -> None:
    archivo.parent.mkdir(parents=True, exist_ok=True)
    temporal = archivo.with_suffix(".tmp")
    with temporal.open("w", encoding="utf-8") as f:
        json.dump(entradas, f, ensure_ascii=False, indent=2)
    os.replace(temporal, archivo)
```

- **Configuración** centralizada en `config.py`: rutas, partida, audio, parámetros de análisis y de puntaje.
- **Logging** con el módulo estándar `logging` hacia `datos/voicer.log`, nunca a la consola, para no ensuciar la interfaz. Se registra el inicio y fin del programa, cada resultado con sus

componentes (útil para calibrar), las advertencias y, en nivel DEBUG, el costo DTW y el largo del camino.

4.13. Decisiones de diseño y trade-offs

Decisión	Alternativas evaluadas	Por qué se eligió
Ranking en un archivo JSON	SQLite (el diseño original), CSV	No hay consultas complejas ni concurrencia: se lee todo, se ordena y se escribe todo. JSON viene con Python, es legible a mano y guarda tipos (int, str) que CSV pierde. SQLite habría sido otra tecnología más que explicar sin aportar nada
MFCC + DTW para el timbre	Comparar espectrogramas completos; promediar los MFCC sin alinear	Los MFCC resumen el timbre en 13 números por cuadro. DTW tolera diferencias de ritmo, que en una imitación son inevitables. Promediar perdería el orden temporal
pyin para el tono	`librosa.yin`, más rápido; `piptrack`	yin no indica qué cuadros no tienen tono, y sin eso un ruido "acertaría" la melodía. pyin es unas 25 veces más lento que los MFCC, pero 190 ms por clip es aceptable en un juego por turnos
Un solo camino de DTW para timbre y tono	Un DTW independiente sobre las curvas de tono	Alinear por timbre es más confiable que por tono, y reutilizar el camino ahorra cómputo y garantiza que se comparan los mismos instantes
Mediana de diferencias de tono	Restar la mediana de cada curva por separado (primera versión)	Ver 4.14: la primera versión se desplazaba con pocos cuadros de más. La mediana de las diferencias, tomadas sobre pares alineados, es robusta a esos cuadros
Curva exponencial para el timbre: $100 \cdot e^{-costo/70}$	Lineal con tope	Nunca da negativo, distingue bien las imitaciones buenas y aplana las malas. La escala 70 se calibró con audios de prueba (Tabla 12)
Ponderación 60 % timbre, 40 % tono	50/50; solo timbre	En voz hablada el timbre es lo que más se percibe; el tono suma en clips con melodía. Es ajustable en config.py
No usar NumPy en nuestro código	Usar NumPy libremente	Restricción propia: que todo el código sea comprensible con el Python visto en clase. Costo: un par de conversiones con `.tolist()`
Precalentar numba al iniciar	No hacer nada	Sin precalentar, el primer jugador esperaba varios segundos después de imitar. Con precalentado, la demora ocurre una vez en el menú, con un mensaje que la explica

Tabla 11. Decisiones técnicas principales.

Calibración del puntaje

Las constantes `ESCALA_COSTO_TIMBRE` = 70 y `ERROR_TONO_MAXIMO` = 4 semitonos se eligieron comparando un fragmento de voz de 3 s de los audios de ejemplo de librosa (`librosa.ex('libri1')`) con variaciones controladas de sí mismo y con otros hablantes. Lo que se buscaba era un orden razonable: el mismo audio deformado debía puntuar más que otra persona diciendo otra cosa, y esto más que un ruido.

Imitación comparada con el original	Costo DTW medio	Timbre	Tono	Total
El mismo audio	0,0	100	100	100
El mismo audio, 25 % más lento	20,7	74	89	80
El mismo audio con ruido de fondo	56,7	45	96	65
El mismo audio, 4 semitonos más grave	61,2	42	78	56
Otra frase del mismo hablante	90,9	27	37	31
Otros hablantes (2 casos)	82,8 – 97,7	25 – 31	32 – 42	27 – 35
Ruido blanco	80,9	32	12	24

Tabla 12. Resultados de la calibración (valores reales obtenidos con nuestro código).

Se observa que DTW absorbe bien la diferencia de velocidad (80 puntos) y que el puntaje de tono es insensible a la altura de la voz pero no al ruido. El caso más débil es el cambio de altura: el timbre cae a 42 porque transponer 4 semitonos también corre los formantes, algo que no ocurre del mismo modo cuando una persona real cambia de registro. Una limitación honesta: estos audios de prueba no son imitaciones humanas, así que la calibración final debería ajustarse jugando (ver 4.14).

Compromisos asumidos

- **Simplicidad frente a precisión.** No usamos modelos entrenados, por ejemplo redes de *embeddings* de voz, que medirían mejor la similitud pero serían una caja negra difícil de explicar y mucho más pesada.
- **Latencia frente a calidad de tono.** Elegir pyin sobre yin cuesta tiempo de análisis a cambio de una decisión de tono confiable.
- **Memoria frente a velocidad.** La caché de análisis guarda unos pocos kilobytes por clip para no repetir cálculos.
- **Portabilidad frente a dependencias.** librosa trae cientos de MB de dependencias para usar seis funciones.

Qué cambiaría si hubiera que escalar o poner en producción

- Ranking compartido entre muchos usuarios → base de datos (SQLite o PostgreSQL) con índices por puntaje, en lugar de reescribir todo el JSON en cada partida, que cuesta O(n).
- Clips largos → DTW con banda (`global_constraints=True`) para bajar el costo cuadrático, o analizar por bloques con `librosa.stream`.

- Muchos jugadores en simultáneo (versión web) → precalcular y guardar el análisis de los clips originales en disco (la caché de librosa o la nuestra serializada) y procesar las imitaciones en procesos aparte.
- Calibración → reunir imitaciones reales puntuadas por personas y ajustar pesos y escalas para maximizar la concordancia con esos puntajes.
- Distribución → empaquetar un ejecutable para Windows, para que los usuarios no tengan que instalar Python.

4.14. Dificultades encontradas y aprendizajes

1. La primera llamada tardaba más de 13 segundos. En la primera prueba, el primer análisis demoró 13,2 s y los siguientes, menos de 0,3 s. La causa es que DTW, Viterbi y partes de pyin están compiladas con numba la primera vez que se usan (sección 4.3). Con `cache=True`, en las ejecuciones siguientes la compilación se lee de disco y el arranque bajó a 1,6 s. La solución fue `precalentar()` en el menú. *Aprendizaje:* una biblioteca en "Python puro" puede tener costos de compilación ocultos.

2. Una grabación de puro ruido no se detectaba como vacía. Esperábamos que `effects.trim` recortara todo si el jugador no decía nada. Al leer el código (`_signal_to_frame_nonsilent` en `effects.py`) vimos que compara cada cuadro contra el **máximo de la propia señal**. En un ruido de fondo uniforme, todos los cuadros están cerca del máximo y nada se recorta. Una señal de ceros exactos tampoco se recorta, como mostró una de nuestras pruebas, que falló en la primera versión. La documentación lo dice (`ref=np.max`), pero la consecuencia no es obvia. Se resolvió agregando antes un control de amplitud absoluta (`PICO_MINIMO`).

3. Unos cuadros de silencio corrían toda la melodía. La primera versión restaba a cada curva de tono su propia mediana. Con un clip de dos notas, una grabación con medio segundo extra mantenía unos cuadros más de la segunda nota, y eso bastaba para mover la mediana de una nota a la otra. La imitación "perfecta" sacaba 61 de tono. La mediana de una distribución con dos grupos parejos es inestable. La solución fue tomar la mediana de las diferencias entre pares ya alineados, y el mismo caso pasó a 99. Agregamos una prueba que reproduce el problema (`test_silencio_al_final_no_corre_la_melodia`).

4. Un ruido "acertaba" la melodía. En la calibración, el ruido blanco sacaba 86 de tono, más que otra persona hablando. pyin marcaba algunos cuadros del ruido como si tuvieran tono, y solo se comparaban esos. Ahora, cuando el original tiene tono y la imitación no, ese paso cuenta como error máximo, y el ruido bajó a 12.

5. Las pruebas fallaban con un error interno de numba. Para que la prueba de una partida completa no imprimiera en pantalla, reemplazamos `builtins.print` por un *mock*. numba falló con `AttributeError: __name__`, porque al compilar resuelve `print` como una función propia y encontró un objeto `Mock`. Se resolvió reemplazando las funciones de `interfaz.py`, lo que de paso confirmó que haber aislado la interfaz era útil.

6. Diferencias entre la documentación que encontrábamos y la versión instalada. Muchos tutoriales y respuestas en foros son de la serie 0.x. En la 1.0 hay que pasar los parámetros por nombre (`librosa.load(ruta, sr=…)`), audioread ya no existe y `set_fftlib` fue reemplazado por el mecanismo de `scipy.fft`. Además, la 1.0 exige Python 3.12, lo que

condiciona la instalación en las computadoras del grupo. *Aprendizaje:* verificar siempre contra la documentación y el código de la versión que se usa.

A completar por el grupo *después de probar con micrófonos reales: qué puntajes obtienen imitaciones humanas buenas y malas, si hubo que reajustar* `ESCALA_COSTO_TIMBRE` *o* `PICO_MINIMO`*, problemas con los micrófonos o permisos de Windows en sus computadoras y qué harían distinto con lo que saben ahora.*

Qué haríamos distinto

- Empezar la calibración con grabaciones humanas reales desde el principio, en lugar de con audios sintéticos y de ejemplo.
- Leer el código fuente de cada función de librosa antes de usarla. Los tres problemas principales se habrían previsto así.
- Escribir las pruebas de casos borde (silencio, ruido, cambios de altura) antes que el código del puntaje.

PARTE III — Conclusiones y cierre

4.15. Conclusiones

Sobre la tecnología

librosa cumplió lo que promete: con unas pocas llamadas obtuvimos representaciones de timbre y de tono y una alineación temporal que, implementadas a mano, habrían ocupado la mayor parte del trabajo. Lo que más valoramos fue poder **leer su código**. Cada algoritmo que usamos (STFT por bloques, MFCC como cadena de transformaciones, DTW y Viterbi como programación dinámica compilada con numba) se puede seguir línea por línea y conectar con los conceptos de la materia: arreglos contiguos frente a listas, tablas de programación dinámica, recorridos hacia atrás, representaciones dispersas de transiciones y la complejidad $O(n \log n)$ de la FFT frente a $O(n^2)$.

También encontramos sus límites. La primera ejecución es lenta por la compilación, la instalación es pesada, y algunas funciones tienen comportamientos que solo se entienden leyendo la implementación. Esto último es una lección general: una biblioteca resuelve el "cómo", pero quien la usa sigue siendo responsable de entender "qué" calcula. Nuestros tres errores principales vinieron de suponer en lugar de verificar.

Sobre el proyecto

El juego funciona de punta a punta: menú, partidas de 1 a 3 jugadores, puntaje, ranking persistente y grabación de clips propios. Tiene 24 pruebas automáticas que pasan y el código respeta el estilo verificado por ruff y black. Separar la interfaz, el audio y el análisis en módulos distintos resultó clave: permitió probar una partida completa sin micrófono y facilitó encontrar el origen de cada problema. El puntaje distingue bien los casos de la calibración, aunque su ajuste fino requiere imitaciones humanas reales.

Posibles mejoras y trabajo futuro

- Calibrar con imitaciones reales puntuadas por personas.
- Mostrar un gráfico de la curva de tono del original y de la imitación al final de cada turno, con `librosa.display`.
- Ponderar el timbre según la probabilidad de tono que ya devuelve pyin (`voiced_prob`).
- Modo "desafío" con dificultad creciente o tiempo límite.
- Interfaz gráfica sencilla (por ejemplo con tkinter, que viene con Python) manteniendo los mismos módulos de lógica.

4.16. División del trabajo

*El grupo definirá el reparto final. La consigna exige que **todos** los integrantes hayan participado en investigación **y** en desarrollo, y que todos expongan.*

Integrante	Investigación	Desarrollo	Documentación	Presentación
Integrante 1				
Integrante 2				
Integrante 3				
Integrante 4				

Tabla 13. División del trabajo (a completar).

4.17. Declaración de uso de herramientas de IA

Se utilizó **Claude** (Anthropic), un asistente de inteligencia artificial, para las siguientes tareas:

- Proponer la idea del proyecto y su alcance a partir de la consigna.
- Generar una primera versión del código del juego y de las pruebas automáticas.
- Buscar y resumir documentación de librosa y leer su código fuente para identificar funciones, estructuras de datos y algoritmos internos.
- Redactar un borrador de este informe y del README, y generar los diagramas.

Qué se verificó contra fuentes primarias: la versión de librosa (1.0.0), su fecha, licencia, requisito de Python y dependencias, en su página de PyPI y con `pip show`. Los nombres de funciones internas, parámetros por defecto y constantes citados en la Parte I se verificaron leyendo el código fuente instalado: `core/audio.py`, `core/spectrum.py`, `core/pitch.py`, `feature/spectral.py`, `sequence.py`, `effects.py`, `_cache.py` y `util/utils.py`. Los tiempos, la calibración y las salidas de ejemplo se obtuvieron ejecutando el código del proyecto. Los algoritmos se contrastaron con sus artículos originales, listados en las referencias.

Pendiente para el grupo: *la consigna exige que todo el contenido pueda ser explicado y defendido por cualquier integrante. Antes de entregar, cada uno debe leer el informe completo, revisar el código, correr el proyecto en su computadora y verificar las referencias (en especial las de la sección 4.5). Ajusten esta declaración para que refleje exactamente cómo usaron la herramienta.*

4.18. Referencias

Formato APA, 7.ª edición. Se priorizaron la documentación oficial, el código fuente y los artículos originales.

Bogdanov, D., Wack, N., Gómez, E., Gulati, S., Herrera, P., Mayor, O., Roma, G., Salamon, J., Zapata, J. y Serra, X. (2013). Essentia: An audio analysis library for music information retrieval. En *Proceedings of the 14th International Society for Music Information Retrieval Conference (ISMIR 2013)* (pp. 493–498).

Brossier, P. M. (2006). *Automatic annotation of musical audio for interactive applications* [Tesis doctoral, Queen Mary University of London]. Biblioteca aubio: https://aubio.org

Cooley, J. W. y Tukey, J. W. (1965). An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, *19*(90), 297–301. https://doi.org/10.1090/S0025-5718-1965-0178586-1

Davis, S. y Mermelstein, P. (1980). Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, *28*(4), 357–366. https://doi.org/10.1109/TASSP.1980.1163420

de Cheveigné, A. y Kawahara, H. (2002). YIN, a fundamental frequency estimator for speech and music. *The Journal of the Acoustical Society of America*, *111*(4), 1917–1930. https://doi.org/10.1121/1.1458024

Geier, M. (2026). *python-sounddevice* (versión 0.5.5) [Software]. https://python-sounddevice.readthedocs.io

Harris, C. R., Millman, K. J., van der Walt, S. J. et al. (2020). Array programming with NumPy. *Nature*, *585*, 357–362. https://doi.org/10.1038/s41586-020-2649-2

Lam, S. K., Pitrou, A. y Seibert, S. (2015). Numba: A LLVM-based Python JIT compiler. En *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC* (pp. 1–6). https://doi.org/10.1145/2833157.2833162

librosa development team. (2026a). *librosa documentation*. https://librosa.org/doc/latest/

librosa development team. (2026b). *librosa 1.0.0* [Paquete de Python]. PyPI. https://pypi.org/project/librosa/1.0.0/

librosa development team. (2026c). *librosa* [Código fuente]. GitHub. https://github.com/librosa/librosa

Mauch, M. y Dixon, S. (2014). pYIN: A fundamental frequency estimator using probabilistic threshold distributions. En *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (pp. 659–663). https://doi.org/10.1109/ICASSP.2014.6853678

McFee, B., Raffel, C., Liang, D., Ellis, D. P. W., McVicar, M., Battenberg, E. y Nieto, O. (2015). librosa: Audio and music signal analysis in Python. En *Proceedings of the 14th Python in Science Conference* (pp. 18–25). https://doi.org/10.25080/Majora-7b98e3ed-003

McFee, B. et al. (2026). *librosa/librosa: 1.0.0* [Software]. Zenodo. https://doi.org/10.5281/zenodo.591533

Müller, M. (2015). *Fundamentals of music processing: Audio, analysis, algorithms, applications*. Springer. https://doi.org/10.1007/978-3-319-21945-5

OpenAI. (2022). *whisper/audio.py* [Código fuente]. GitHub. https://github.com/openai/whisper/blob/main/whisper/audio.py

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C. y Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. En *Proceedings of the 40th International Conference on Machine Learning* (pp. 28492–28518). PMLR.

Sakoe, H. y Chiba, S. (1978). Dynamic programming algorithm optimization for spoken word recognition. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, *26*(1), 43–49. https://doi.org/10.1109/TASSP.1978.1163055

Virtanen, P., Gommers, R., Oliphant, T. E. et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*, 261–272. https://doi.org/10.1038/s41592-019-0686-2

Yang, Y.-Y., Hira, M., Ni, Z. et al. (2022). TorchAudio: Building blocks for audio and speech processing. En *ICASSP 2022* (pp. 6982–6986). https://doi.org/10.1109/ICASSP43922.2022.9747236

Anexos

Anexo A. Salida de las pruebas automáticas

Resultado de `python -m unittest discover -s tests -v` (entre corchetes, el archivo de prueba):

```
test_elegir_sin_repetir  [packs] ... ok
test_listar_clips_ignora_otros_archivos  [packs] ... ok
test_nombre_de_archivo_seguro  [packs] ... ok
test_nombre_legible  [packs] ... ok
test_nombre_libre_no_pisa  [packs] ... ok
test_pack_vacio_o_inexistente  [packs] ... ok
test_solo_lista_packs_con_wav  [packs] ... ok
test_partida_de_dos_jugadores  [partida] ... ok
test_mfcc_sin_coeficiente_cero  [puntaje] ... ok
test_semitonos_sin_tono  [puntaje] ... ok
test_silencio_es_grabacion_vacia  [puntaje] ... ok
test_tono_en_semitonos  [puntaje] ... ok
test_misma_senal_da_100 (test_puntaje.TestComparacion.test_misma_senal_da_100) ... ok
test_parecido_gana_a_distinto  [puntaje] ... ok
test_puntajes_en_rango  [puntaje] ... ok
test_velocidad_distinta_no_castiga_tanto  [puntaje] ... ok
test_imitacion_muda_da_cero_de_tono  [puntaje] ... ok
test_original_sin_tono_no_evalua_tono  [puntaje] ... ok
test_otra_altura_misma_melodia  [puntaje] ... ok
test_silencio_al_final_no_corre_la_melodia  [puntaje] ... ok
test_archivo_corrupto_se_respalda  [ranking] ... ok
test_archivo_inexistente_da_lista_vacia  [ranking] ... ok
test_entradas_invalidas_se_ignoran  [ranking] ... ok
test_registrar_y_ordenar  [ranking] ... ok
Ran 24 tests in 3.070s
OK
```

Anexo B. Constantes de configuración

```
# --- Partida ---------------------------------------------------------------
MAX_JUGADORES = 3
RONDAS_POR_PARTIDA = 3
LARGO_MAXIMO_NOMBRE = 15
SEGUNDOS_CUENTA_REGRESIVA = 3
SEGUNDOS_EXTRA_GRABACION = 1.0  # margen por si el jugador arranca tarde
SEGUNDOS_CLIP_NUEVO = 4.0  # duración al grabar un clip propio
GUARDAR_GRABACIONES = True  # guarda cada imitación en grabaciones/

# --- Audio -----------------------------------------------------------------
FRECUENCIA_MUESTREO = 22050  # Hz; la misma que usa librosa por defecto
UMBRAL_SILENCIO_DB = 30  # para librosa.effects.trim
DURACION_MINIMA_SEG = 0.3  # por debajo de esto la grabación se considera vacía
PICO_MINIMO = 0.02  # amplitud máxima (de 0 a 1) por debajo de la cual es silencio

# --- Análisis con librosa --------------------------------------------------
TAMANIO_VENTANA = 2048  # muestras por cuadro (n_fft / frame_length)
SALTO = 512  # hop_length: mismo valor para MFCC y pyin
CANTIDAD_MFCC = 14  # se calculan 14 y se descarta el coeficiente 0
NOTA_MINIMA = "C2"  # rango de búsqueda de pyin (unos 65 Hz)
NOTA_MAXIMA = "C7"  # (unos 2093 Hz, alcanza para silbidos)

# --- Puntaje ---------------------------------------------------------------
PESO_TIMBRE = 0.6
PESO_TONO = 0.4
ESCALA_COSTO_TIMBRE = 70.0  # calibrado con audios de prueba (ver informe, 4.13)
ERROR_TONO_MAXIMO = 4.0  # semitonos de error medio que dan 0 puntos de tono
PARES_SONOROS_MINIMOS = 5  # cuadros con tono en ambos audios para evaluar el tono
```

Anexo C. Glosario

Término	Significado
Frecuencia de muestreo	Cantidad de valores por segundo con que se digitaliza el sonido (22 050 Hz en el juego)
Cuadro (frame)	Ventana corta de la señal (2048 muestras, unos 93 ms) que se analiza como una unidad; los cuadros avanzan de a 512 muestras (unos 23 ms)
Espectrograma	Matriz que indica cuánta energía hay en cada frecuencia en cada instante
Escala mel	Escala de frecuencias que imita la percepción humana: más detalle en graves que en agudos
MFCC	Coeficientes que resumen la forma del espectro de un cuadro; describen el timbre
Frecuencia fundamental (f0)	La frecuencia que percibimos como la altura o nota de un sonido
Semitono	La menor distancia entre dos notas del piano; 12 semitonos forman una octava
DTW	Alineación de dos secuencias que permite estirar o comprimir el tiempo
HMM / Viterbi	Modelo oculto de Markov y el algoritmo que encuentra su secuencia de estados más probable
JIT	Compilación "justo a tiempo": el código se traduce a lenguaje de máquina al momento de usarse

Tabla 14. Glosario de términos técnicos.